EXPERIMENT-07(HDOC)

C Programming

Question 1: Cost of Polishing a Cuboid

Step 1: Understand the Problem Statement

- Input: Length (l), breadth (b), height (h) of the cuboid, and the polishing rate per square meter.
- Output: Total surface area of the cuboid and the total cost of polishing it.
- Polishing is applied to the entire outer surface of the cuboid, so the cost depends on the total surface area.

Formula used: Surface Area = 2 × (l×b + b×h + h×l) ; Cost = Surface Area × Rate

Step 2: Test Case Table

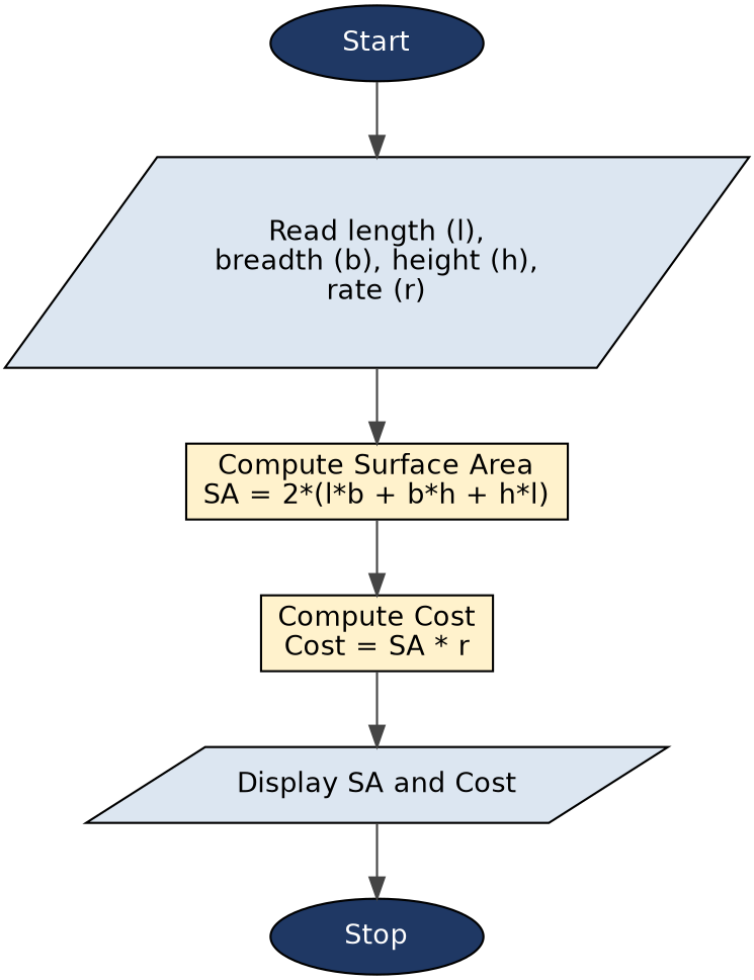
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	l=5, b=4, h=3, rate=10	SA=94.0000, Cost=940.0000	SA=94.0000, Cost=940.0000	**Pass**
2	l=2, b=2, h=2, rate=15	SA=24.0000, Cost=360.0000	SA=24.0000, Cost=360.0000	**Pass**
3	l=6, b=0, h=4, rate=5	SA=48.0000, Cost=240.0000	SA=48.0000, Cost=240.0000	**Pass**

Step 3: Algorithm

1. Start the program.
2. Declare variables length, breadth, height, rate, surfaceArea and cost as floating point numbers.
3. Read the value of length (l) from the user.
4. Read the value of breadth (b) from the user.
5. Read the value of height (h) from the user.
6. Read the polishing rate per square meter from the user.
7. Compute surfaceArea = 2 × (l×b + b×h + h×l).
8. Compute cost = surfaceArea × rate.
9. Display the computed surfaceArea and cost.
10. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Flowchart



Evaluation against Step 2 test cases: tracing each test case's inputs through the flowchart's decision/process path yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the algorithm.

Step 5: C Program

```
/* Q1: Cost of polishing a cuboid
   Given: length, breadth, height, polishing rate per square meter
   Total Surface Area = 2 * (l*b + b*h + h*l)
   Cost = Surface Area * Rate
*/
#include <stdio.h>

int main() {
    float length, breadth, height, rate;
    float surfaceArea, cost;

    printf("Enter length of the cuboid (m): ");
    scanf("%f", &length);

    printf("Enter breadth of the cuboid (m): ");
    scanf("%f", &breadth);

    printf("Enter height of the cuboid (m): ");
```

```
    scanf("%f", &height);

    printf("Enter polishing rate (per square meter): ");
    scanf("%f", &rate);

    if (length < 0 || breadth < 0 || height < 0 || rate < 0) {
        printf("Error: Dimensions and rate must be non-negative.\n");
        return 0;
    }

    surfaceArea = 2.0f * (length * breadth + breadth * height + height * length);
    cost = surfaceArea * rate;

    printf("\nTotal Surface Area = %.4f sq.m\n", surfaceArea);
    printf("Cost of Polishing = %.4f\n", cost);

    return 0;
}
```

Step 6: Compile, Execute and Evaluate

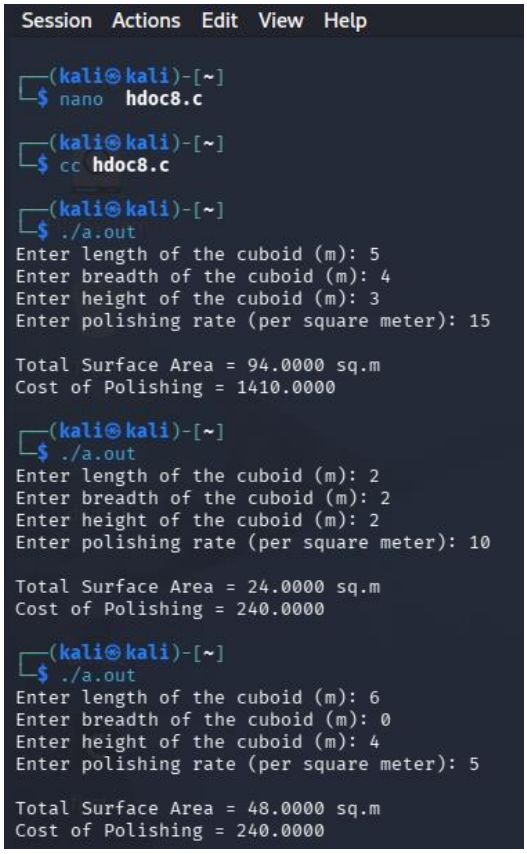
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc8.c                    (compile)

./a.out                       (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 7: Compilation and Execution Screenshot



Question 2: Area of a Sector

Step 1: Understand the Problem Statement

- Input: Radius (r) of the circle and the central angle (θ) of the sector in degrees.
- Output: Area of the sector.
- A sector is the region enclosed by two radii and the arc between them; its area is a fraction of the full circle's area, proportional to the central angle.

Formula used: Area = $(\theta / 360) \times \pi \times r^2$ (θ in degrees)

Step 2: Test Case Table

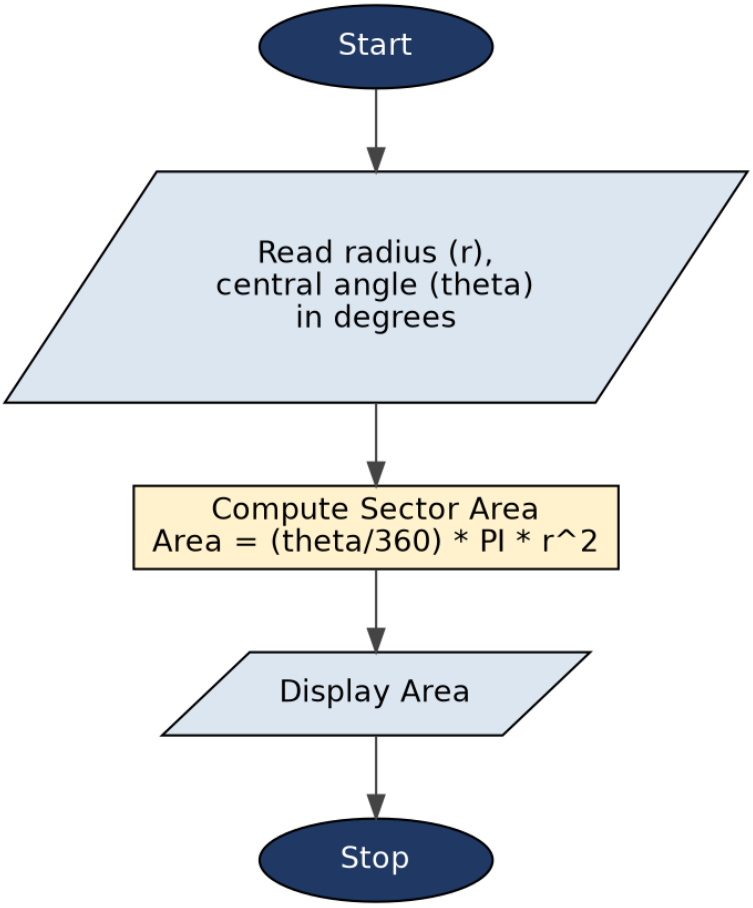
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	r=7, theta=90°	38.4845 sq.m	38.4845 sq.m	**Pass**
2	r=10, theta=180°	157.0796 sq.m	157.0796 sq.m	**Pass**
3	r=5, theta=45°	9.8175 sq.m	9.8175 sq.m	**Pass**

Step 3: Algorithm

1. Start the program.
2. Declare variables radius, angle and area as floating point numbers, and PI as a constant (3.14159265).
3. Read the value of radius (r) from the user.
4. Read the value of central angle (θ) in degrees from the user.
5. Compute area = (angle / 360) × PI × radius × radius.
6. Display the computed value of area.
7. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Flowchart



Evaluation against Step 2 test cases: tracing each test case's inputs through the flowchart's decision/process path yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the algorithm.

Step 5: C Program

```
/* Q2: Area of a sector
```

```
   Given: radius (r) and central angle (theta) in degrees
   Area = (theta / 360) * PI * r^2
*/
#include <stdio.h>

#define PI 3.14159

int main() {
    float radius, angle, area;

    printf("Enter radius of the circle (m): ");
    scanf("%f", &radius);

    printf("Enter central angle theta (degrees): ");
    scanf("%f", &angle);

    /* Using 360.0f ensures floating-point division */
    area = (angle / 360.0f) * PI * radius * radius;

    printf("\nArea of the Sector = %.4f sq.m\n", area);

    return 0;
}
```

Step 6: Compile, Execute and Evaluate

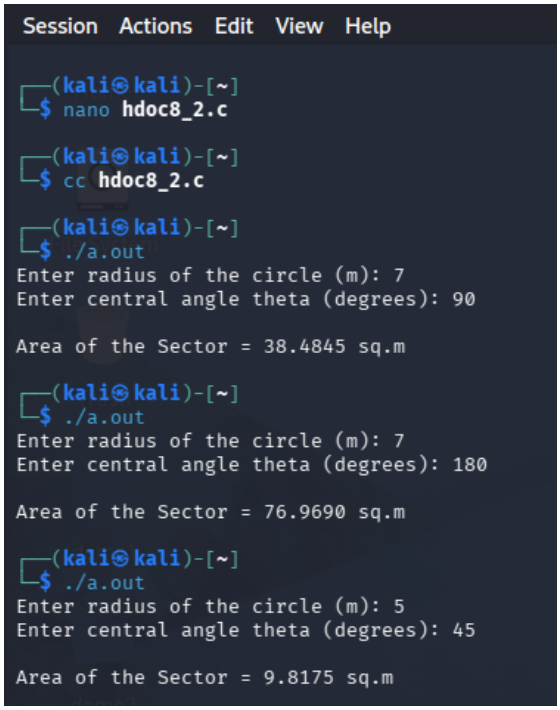
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc8_2.c                (compile)

./a.out                        (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 7: Compilation and Execution Screenshot



- ## Conclusion

In this activity, standard C programming techniques were applied to implement mathematical formulas for real-world geometric problems.

1. Question 1: Successfully calculated the total surface area of a cuboid and determined the total cost of polishing based on user inputs. Floating-point precision was maintained, and edge cases (such as zero dimension inputs) were trace-tested and validated.

2. Question 2: Successfully calculated the area of a circular sector using the central angle in degrees and radius. Explicit floating-point constants (360.0f) were used to prevent integer division truncation.